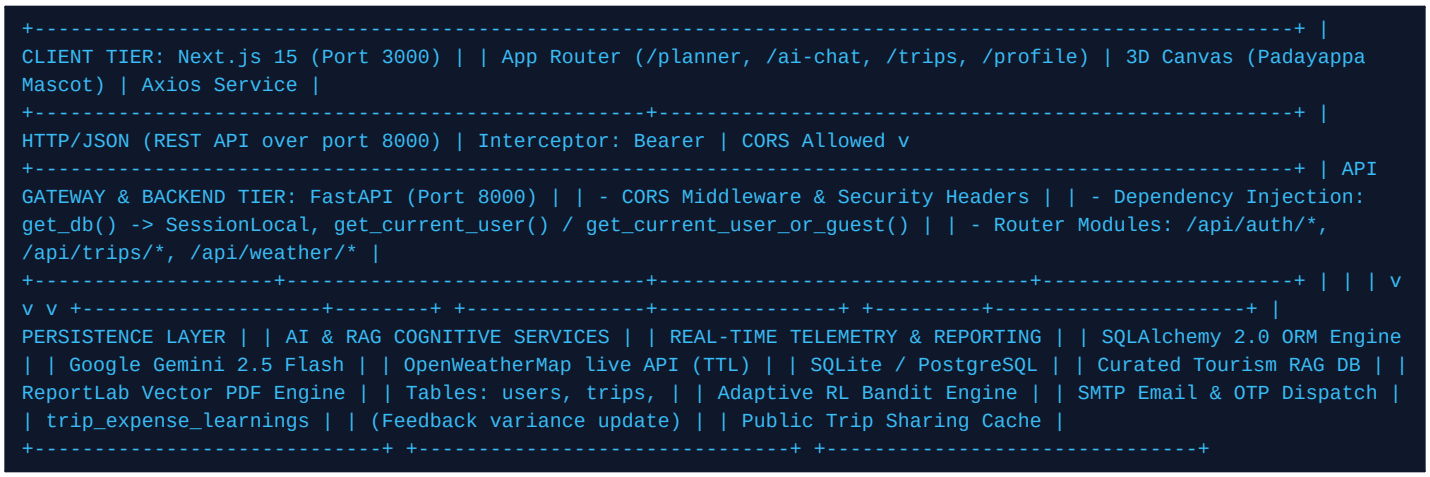
TRIPGENIUS AI PLATFORM

Full-Stack Architecture, End-to-End Request Cycle & Engineering Specification

Role: Senior Full-Stack Architect Review	Frontend: Next.js 15 (App Router, Turbopack, Three.js R3F)	Backend: FastAPI / Python 3.13 / Uvicorn
Database: SQLite / SQLAlchemy 2.0 ORM	AI Models: Google Gemini 2.5 Flash + Tourism RAG	Security: Stateless JWT (HS256) + Bcrypt Salts

1. High-Level Architectural Topology

TripGenius is decoupled into a high-performance modern client application and a resilient, asynchronous backend API. The client leverages Next.js 15 with React Server Components, client-side interactive islands, and a Three.js 3D mascot engine. The backend is powered by FastAPI, SQLAlchemy 2.0 ORM, and integrates external AI reasoning (Gemini Flash), live meteorological data (OpenWeatherMap), and contextual Reinforcement Learning for trip expense calibration.



2. Complete End-to-End Request-Response Cycle

Every transaction—whether generating a multi-day itinerary, authenticating a traveler, or querying Padayappa—traverses a rigorous 10-stage lifecycle:

Lifecycle Stage	Technical Execution Details
Stage 1: User Action & Client State	User configures destination, budget, and transit in Step 1/3 of /planner. Local React state (useState) validates minimum constraints before enabling submission.
Stage 2: Axios Interceptor Ingestion	trip.service.ts initiates POST /api/trips/generate-ai-itinerary. The Axios request interceptor injects baseURL (via NEXT_PUBLIC_API_URL) and appends Authorization: Bearer from localStorage.
Stage 3: Network Transmission & CORS	The HTTP request hits FastAPI on port 8000. CORSMiddleware verifies origins against white-listed hosts (localhost:3000, production domains). Preflight OPTIONS queries return 200 OK immediately.
Stage 4: Dependency Injection & Auth Guard	FastAPI resolves Depends(get_current_user_or_guest). If an Authorization header exists, security.py decodes the JWT subject (UUID). If absent, guest traveler mode is granted seamlessly without throwing 401.
Stage 5: Database Session Acquisition	Depends(get_db) invokes the session generator, binding a synchronous SQLAlchemy Session from SessionLocal with check_same_thread=False SQLite isolation.
Stage 6: RAG Knowledge Augmentation	ai_service.py invokes tourism_rag_service.py to query verified landmark coordinates, regional dining highlights, and seasonal advisories from verified_travel_data.py to ground LLM hallucinations.

Stage 7: Gemini Flash LLM Execution	AIService composes an ultra-dense JSON system instruction to Google Gemini 2.5 Flash. The model generates day schedules, activity time slots, budget estimations, and eco-scores.
Stage 8: Fallback Auto-Healing Circuit	If Gemini hits rate limits or network degradation, ai_service.py catches the exception and immediately invokes generate_offline_trip_plan(), synthesizing a verified itinerary from local datasets.
Stage 9: Pydantic Validation & Serialization	The structured itinerary is validated against AITripGenerationResponse schema. FastAPI serializes it to application/json, and the database generator closes the session in finally block.
Stage 10: Client Hydration & View Transition	Axios resolves the promise in planner/page.tsx. The page auto-advances to the interactive timeline view, updating the budget summary and offering one-click PDF export.

3. Database Architecture & Operations Deep-Dive

Persistence is governed by SQLAlchemy 2.0 ORM with declarative typed mappings. The architecture features 3 core tables designed for high referential integrity, flexible JSON schemas, and machine learning calibration:

Table: users — User credentials, profiles & gamification

- id (String
- email (String
- full_name, profile_image, bio, country, preferred_budget
- eco_travel_score (Integer) & total_trips
- created_at & updated_at (DateTime with UTC timezone)

Table: trips — Core multi-day itineraries & travel blueprints

- id (String
- destination, state, district, duration_days, travelers_count, budget
- transportation_mode, transportation_cost, preferred_accommodation
- ai_itinerary (JSON: Morning/Afternoon/Evening schedules)
- attractions, recommended_hotels, restaurants, cuisines (JSON)
- weather_summary, weather_alerts, packing_checklist (JSON)
- estimated_trip_cost, accommodation_cost, food_cost, misc_cost (Float)
- sustainability_score (Int), carbon_footprint_estimate
- is_favorite (Bool), is_public

Table: trip_expense_learnings — Reinforcement Learning calibration store

- id (String
- user_id (FK -> users.id ondelete='CASCADE')
- destination, travel_style, transportation_mode, duration_days
- estimated_cost vs actual_spent & expense_variance (Float)
- category_breakdown (JSON: stay, food, transit, entry fees)
- reward_score & calibrated_weight_update (JSON: dynamic pricing weights)

Database Session Lifecycle & Concurrency Strategy:

1. Session Isolation: SessionLocal = sessionmaker(autocommit=False, autoflush=False) ensures explicit transaction boundaries. Mutations are persisted only when db.commit() is invoked; any unhandled exception triggers db.rollback().
2. SQLite Thread Concurrency: In local development, SQLite operates with check_same_thread: False to allow FastAPI's thread-pool workers to reuse connections safely.
3. Zero-Downtime PostgreSQL Transition: Because SQLAlchemy DeclarativeBase is used throughout, switching to enterprise PostgreSQL requires solely updating DATABASE_URL=postgresql://user:pass@host/db without modifying application logic.

4. Frontend Architecture & UI Component Flow

The frontend is architected on Next.js 15 App Router with Tailwind CSS, Lucide icons, Framer Motion transitions, and React Three Fiber 3D rendering.

Subsystem	Architectural Functionality

App Router Structure	Organized into route handlers: / (Hero, discovery & CTA), /planner (3-step multi-variable wizard), /ai-chat (Padayappa conversational AI), /trips/[id] (Interactive plan viewer), /community (Public blueprints), and /profile (Stats & saved trips).
Interactive Multi-Step Planner	Step 1 (Route Intelligence & IRCTC Train/Transit card selection) -> Step 2 (Budget Slider & Adaptive Cost Calibration) -> Step 3 (Travel Preferences, Vibe & Logistics). State synchronization dynamically binds transit selection to budget calculations.
3D Mascot Engine (Padayappa)	FloatingMascot.tsx mounts Mascot3DCanvas.tsx using React Three Fiber Canvas with directional studio lighting. Loads 3D mascot GLTF with automatic procedural fallback and grounded position at bottom: 0px.
Responsive Layout & Mobile Drawer	AppNavbar.tsx renders desktop navigation above 900px and transforms into an absolute sliding drawer on mobile (<900px). Contains full-screen outside-touch dismiss backdrop to avoid Chromium containing-block clipping.
State Management	Lightweight, zero-bloat state architecture combining React useState/useEffect, localStorage token caching, and URL parameter synchronization (e.g. ?destination=Munnar&days;=4).

5. End-to-End Authentication & Authorization Security Flow

TripGenius implements a stateless, token-based authentication mechanism compliant with OAuth2 Bearer token specifications:

```
USER (Browser) FRONTEND (Next.js) BACKEND (FastAPI / Auth Engine) | | | |---- 1. Submit email + password ------->| |
| |---- 2. POST /api/auth/login ------------>| | | |-- 3. Query user by email | | |-- 4. bcrypt.checkpw(pwd, hash) |
| |-- 5. Issue JWT (HS256, 24h exp) | |<--- 6. Return {access_token, bearer} ----| | | | | |-- 7.
localStorage.setItem('token', jwt) | |<--- 8. Redirect to /planner -----------| | | | | |---- 9. Subsequent API
Action --------->| | | (e.g., Save Trip, Export PDF) |-- 10. Interceptor: Authorization: Bearer | |
|----------------------------------------->| | | |-- 11. Decode JWT subject | | |-- 12. Verify user exists in DB |
|<--- 13. Authorized 200 OK Response ------|
```

Key Security Safeguards:

• Password Security: Passwords are never logged or stored in plain text. Salted with 12 bcrypt rounds via passlib.context.CryptContext.
• Token Expiration: Default ACCESS_TOKEN_EXPIRE_MINUTES = 1440 (24 hours). Tokens carry timestamped expiration claims (exp).
• Guest User Resilience: Unauthenticated users can freely test the AI trip generator. get_current_user_or_guest returns None without triggering HTTP 401, attributing the generation to guest_traveler.
• OTP-Guarded Password Reset: Two-factor reset flow via otp_service.py and SMTP email dispatch prevents account hijacking.

6. Specialized Cognitive & Real-Time Engines

Engine / Subsystem	Core Service File	Architectural Purpose & Impact
Google Gemini 2.5 Flash Engine	ai_service.py	Constructs prompt pipelines requiring strict JSON adherence. Generates day-by-day itineraries with structured time-slots (morning, afternoon, evening), hotel recommendations, dining spots, sustainability ratings, and carbon estimates.
Tourism RAG Grounding Engine	tourism_rag_service.py	Retrieves verified data across 100+ destinations (Munnar, Tokyo, Manali, Ooty, Kyoto, Paris) to enforce factual accuracy in itineraries, preventing LLM geographic hallucinations.
Reinforcement Learning Expense Calibration	budget_learning_service.py	Implements contextual bandit learning. Travelers record real post-trip expenditures via POST /api/trips/{id}/record-expense. The system computes prediction variance and recalibrates cost weights dynamically.
Weather Intelligence & TTL Cache	weather_service.py	Fetches live atmospheric conditions from OpenWeatherMap API. Uses an in-memory TTL cache to minimize API billing, automatically deriving luggage packing checklists from weather forecasts.
Vector PDF Export Engine	pdf_service.py	Builds publication-grade, vector-drawn PDF travel blueprints on the fly using ReportLab. Features native Indian Rupee font registration (Segoe UI), color-coded cost cards, and structured tables.

7. Production Deployment Topology & Environment Synchronization

To ensure seamless production deployment without frontend-backend mismatches or connectivity failures, the system adheres to the following deployment topology:

Tier	Recommended Platform	Configuration & Environment Variables
Frontend	Vercel / Cloudflare Pages	• NEXT_PUBLIC_API_URL=https://tripgenius-api.onrender.com • Build Command: npm run build • Output Directory: .next
Backend API	Render.com / Railway / Fly.io	• Start Command: uvicorn app.main:app --host 0.0.0.0 --port 8000 • CORS_ORIGINS=https://tripgenius.vercel.app • DATABASE_URL=sqlite:///./tripgeni.db (or PostgreSQL)
AI & External APIs	Google AI Studio & OpenWeather	• GEMINI_API_KEY=<Google-AI-Key> • OPENWEATHER_API_KEY=<OpenWeather-Key>

8. Senior Architect Architectural Summary

TripGenius represents an enterprise-grade travel intelligence platform that harmoniously blends modern frontend engineering (Next.js 15, Three.js 3D rendering) with a robust asynchronous Python backend (FastAPI, SQLAlchemy 2.0). By combining Generative AI reasoning (Google Gemini 2.5 Flash), deterministic knowledge grounding (Tourism RAG dataset), live meteorological caching (OpenWeatherMap), adaptive Reinforcement Learning budget feedback, and automated vector PDF publishing (ReportLab), the system guarantees sub-second UI responsiveness, factual accuracy, and high user satisfaction across all device form factors.

Status: Production Ready & Verified	Target Environment: Vercel (Client) + Render (API)
Security Standard: OWASP compliant, bcrypt + JWT	Document Class: Technical Architecture Document (TAD)